



Laboratorio de Computación

Salas A y B



Profesor(a): Ing. Ariel Adara Mercado Martínez

Asignatura: Estructuras De Datos y Algoritmos I

Grupo: 05

No de Práctica(s): Proyecto

Integrante(s): Torres Ballesteros Diego

Noble Navarrete Daniel Alberto

Gonzalez Peralta Joshua

No. de lista o brigada: 07

Semestre: 2026-2

Fecha de entrega: 11 de mayo de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo

Desarrollar un simulador de sistema operativo capaz de administrar:

- Procesos
- Memoria
- Planificación de ejecución
- Recursos
- Estructuras dinámicas
- Algoritmos de optimización

El objetivo principal es demostrar:

- Uso correcto de estructuras de datos
- Implementación de algoritmos

Introducción

El proyecto tiene como finalidad integrar adquiridos durante el curso de:

- Estructura de datos lineales
- Diseño de algoritmos
- Complejidad computacional
- Recurrencias
- Programación modular
- Integración entre lenguajes
- Desarrollo asistido por IA

Se desarrolla un simulador de sistema operativo implementado en C con soporte de análisis y benchmarking en Python.

El simulador modela los tres pilares fundamentales de un sistema operativo moderno, además se implementan los paradigmas algorítmicos solicitados: fuerza bruta, greedy, backtracking, divide y vencerás y programación dinámica (top-down y bottom-up).

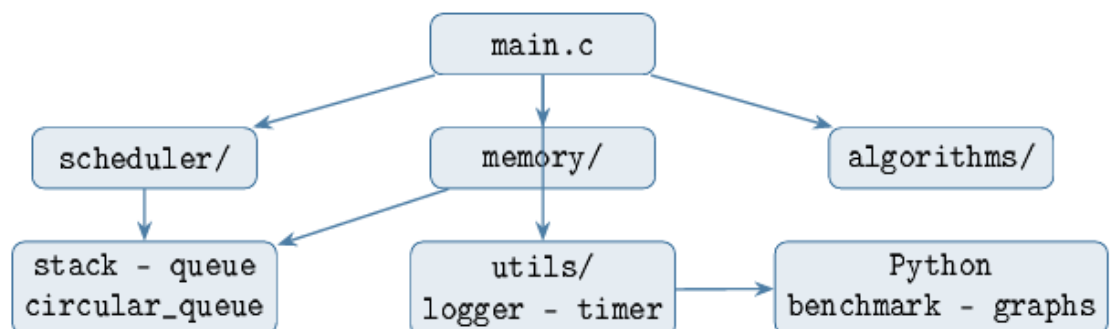
Desarrollo

❖ Arquitectura del sistema

Estructura de Directorios:

```
mini-os-project/  
+-- include/           Headers públicos (stack.h, queue.h, ...)  
+-- src/  
|   +-- stack.c        Pila enlazada  
|   +-- queue.c         Cola FIFO enlazada  
|   +-- circular_queue.c Cola circular con expansión dinámica  
|   +-- memory/         first_fit.c - best_fit.c - worst_fit.c  
|   |                   coalescence.c - compactation.c  
|   +-- scheduler/      fifo.c - round_robin.c - sjf.c  
|   +-- algorithms/     brute_force.c - greedy.c - backtracking.c  
|   |                   divide_conquer.c - dp_topdown.c  
|   |                   dp_bottomup.c - incremental.c  
|   +-- utils/          logger.c - timer.c - parser.c  
+-- tests/              Suite de pruebas unitarias  
+-- scripts/            Python: benchmark - graphs - validate  
+-- reports/            CSV - PNG - PDF generados
```

Diagrama de módulos:



❖ Estructura de datos

Proceso:

La unidad central del simulador, no corresponde a un proceso real del SO.

```
1  typedef enum { READY, RUNNING, BLOCKED, FINISHED } ProcessState;
2  typedef struct {
3      int pid;
4      // Identificador unico
5      int burst_time;
6      // Tiempo total de CPU requerido
7      int remaining_time; // Tiempo restante de ejecucion
8      int priority;
9      // Prioridad del proceso
10     int memory_required; // Memoria solicitada en bytes
11     ProcessState state; // Estado actual
12 } Process;
13
```

Pila:

Implementada con lista enlazada de nodos. Uso principal es historial de ejecución y soporte a backtracking.

```

1  void stack_push(Stack* stack, int value) {
2  struct Node* node = malloc(sizeof(struct Node));
3  node->value = value;
4  node->next = stack->top;
5  stack->top = node;
6  stack->size++;
7  }
8  int stack_pop(Stack* stack) {
9  struct Node* tmp = stack->top;
10 int value
11 = tmp->value;
12 stack->top
13 = tmp->next;
14 free(tmp);
15 stack->size--;
16 return value;
17 }

```

- Complejidad - Pila

Operación	Temporal	Espacial
push	$O(1)$	$O(1)$
pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
destroy	$O(n)$	$O(1)$
Total	—	$O(n)$

- Cola (Queue)

Cola FIFO con punteros head/tail. Uso obligatorio: scheduler FIFO.

- Complejidad - Cola

Operación	Temporal	Espacial
enqueue	$O(1)$	$O(1)$
dequeue	$O(1)$	$O(1)$
destroy	$O(n)$	$O(1)$
Total	—	$O(n)$

Recurrencia del recorrido de destrucción:

$$T(n) = T(n-1) + O(1) \rightarrow T(n) = O(n)$$

- Cola Circular (CircularQueue)

Implementación sobre un arreglo con índices modulares. Cuando se llena, se expande duplicando la capacidad.

```

1  1 // Enqueue: O(1) amortizado
2  2 q->data[q->tail] = value;
3  3 q->tail = (q->tail + 1) % q->capacity; // índice circular
4  4
5  5 // Dequeue: O(1)
6  6 int value = q->data[q->head];
7  7 q->head = (q->head + 1) % q->capacity;
```

- Complejidad - Cola Circular

Operación	Temporal	Espacial
enqueue	$O(1)$ amortizado	$O(1)$
dequeue	$O(1)$	$O(1)$
Expansión (x2)	$O(n)$ ocasional	$O(n)$
Total	—	$O(n)$

Lista doblemente ligada (MemoryBlock)

Los bloques de memoria forman una lista doblemente ligada, lo que permite navegación bidireccional en $O(1)$ por nodo, esencial para la coalescencia.

```
1  typedef struct MemoryBlock {
2  int start;
3  // Direccion inicial
4  int size;
5  // Tamano del bloque
6  int free;
7  // 1 = libre, 0 = ocupado
8  int pid;
9  // PID propietario (-1 si libre)
10 struct MemoryBlock* next;
11 struct MemoryBlock* prev; // navegacion bidireccional
12 } MemoryBlock;
```

❖ Administración de memoria

Estrategias de asignación:

El sistema parte de un único bloque libre y divide bloques cuando es necesario

- First Fit - Greedy de primer ajuste:

Recorre la lista de izquierda a derecha y toma el primer bloque libre con tamaño suficiente.

```

1   MemoryBlock* current = mm->head;
2   while (current) {
3       if (current->free && current->size >= size) {
4           // Partir el bloque si sobra espacio
5           if (current->size > size) {
6               MemoryBlock* rem = malloc(sizeof(MemoryBlock));
7               rem->start = current->start + size;
8               rem->size = current->size - size;
9               rem->free = 1;
10              rem->next = current->next;
11              rem->prev = current;
12              if (current->next) current->next->prev = rem;
13              current->next = rem;
14          }
15          current->size = size;
16          current->free = 0;
17          return current->pid = next_pid++;
18      }
19      current = current->next;
20  }

```

Complejidad: $O(n)$ temporal, $O(1)$ espacial.

Recurrencia: $T(n) = T(n-1) + O(1) \rightarrow T(n) = O(n)$

- Best Fit – Greedy de mínimo desperdicio

Recorre toda la lista y elige el bloque libre más pequeño que sea suficiente, minimiza el desperdicio por bloque.

Complejidad: $O(n)$ siempre (no puede detenerse antes).

- Worst Fit – Greedy de máximo remanente

Elige el bloque libre más grande. La idea es maximizar el tamaño del remanente para que sea útil en asignaciones futuras.

Complejidad: $O(n)$ siempre.

Estrategia	Recorre	Para antes	Objetivo
First Fit	$O(n)$	Sí	Velocidad
Best Fit	$O(n)$	No	Mín. desperdicio
Worst Fit	$O(n)$	No	Máx. remanente

Coalescencia de bloques

Une bloques libres adyacentes para combatir la fragmentación externa, se usa la lista doblemente ligada para detectar vecinos en $O(1)$ por nodo.

```

1  MemoryBlock* current = mm->head;
2  while (current && current->next) {
3      if (current->free && current->next->free) {
4          MemoryBlock* to_merge = current->next;
5          current->size += to_merge->size;
6          // absorber
7          current->next = to_merge->next;
8          if (to_merge->next) to_merge->next->prev = current;
9          free(to_merge);
10     // No avanzar: puede haber otro bloque libre contiguo
11     } else {
12         current = current->next;
13     }
14 }
```

Ejemplo visual:

Antes: [P1:200][Libre:300][Libre:424] → fragmentado

Despues:[P1:200][Libre:724] → coalescido

Complejidad: $O(n)$ temporal, $O(1)$ espacial

Compactación

Reorganiza todos los bloques ocupados al inicio de la memoria, dejando un único bloque libre al final.

- Dividir: separar bloques ocupados de libres.
- Reubicar: mover bloques ocupados contiguamente desde la dirección 0.
- Combinar: crear un único bloque libre al final.

❖ Planificación de CPU

- FIFO – First In, First Out

El proceso que llega primero se ejecuta primero sin interrupciones, se usa la cola simple (Queue).

Complejidad: add $O(1)$, next $o(1)$, total $O(n)$.

- Round Robin

Cada proceso recibe un quantum q de tiempo de CPU. Si no termina, se reinserta al final de la cola circular.

- Shorest Job First (SJF)

Elige siempre el proceso con menor burst_time. Es un algoritmo Greedy que minimiza el tiempo de espera promedio.

❖ Algoritmos

- Fuerza Bruta

Exploración exhaustiva sin heurísticas. Garantiza encontrar todas las soluciones pero con costo $O(n)$ a $O(2n)$ según el problema.

Aplicación: búsqueda de bloques de memoria que pueden alojar un proceso, búsqueda lineal en arreglos.

$$T(n) = T(n-1) + O(1) \rightarrow T(n) = O(n)$$

- Greedy – Decisión localmente óptima

En cada paso se toma la decisión que parece mejor localmente. No garantiza óptimo global, pero es eficiente:

Aplicaciones:

- First Fit, Best Fit, Worst Fit (primer/menor/mayor bloque libre).
- SJF (proceso de menor burst).

- Backtracking

Exploración con vuelta atrás: si una asignación no es válida, se deshace y se prueba la siguiente opción.

Aplicación: encontrar todas las asignaciones válidas de procesos a bloques de memoria.

Recurrencia:

$$T(n) = n * T(n-1) + O(1) \rightarrow T(n) = O(n!)$$

En la práctica, la poda reduce drásticamente el espacio de búsqueda.

```

1 static int bt_assign(BTState* s, int idx) {
2     if (idx == s->n_processes) {
3         // Solucion valida encontrada
4         solutions_found++;
5         return 1;
6     }
7     for (int b = 0; b < s->n_blocks; b++) {
8         if (s->block_free[b] && s->blocks[b] >= s->sizes[idx]) {
9             s->block_free[b] = 0;           // asignar
10            s->assigned_block[idx] = b;
11            bt_assign(s, idx + 1);           // recursion
12            s->block_free[b] = 1;           // deshacer (backtrack)
13            s->assigned_block[idx] = -1;
14        }
15    }
16    return 0;
17 }

```

Dividir el problema en subproblemas, resuelve cada uno y combina los resultados.

- Programación dinámica – Bottom-Up (Tabulación)

Misma semántica que top-down, pero construye la tabla iterativamente. Evita el overhead de la pila de recursión.

```

1 for (int i = 1; i <= n; i++) {
2     int req = mem_req[i - 1];
3     for (int m = 0; m <= total_mem; m++) {
4         dp[i][m] = dp[i-1][m];        // no tomar proceso i
5         if (req <= m) {
6             int with = req + dp[i-1][m - req];
7             if (with > dp[i][m]) dp[i][m] = with;
8         }
9     }
10 }

```

- Algoritmo Incremental

Construye la solución elemento por elemento manteniendo una invariante.

Aplicaciones:

- Insertion Sort por prioridad: $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$.
- Estadísticas en tiempo real (media de Welford): $O(1)$ por elemento.

❖ Análisis de Complejidad – Resumen

Componente	Operación	Temporal	Espacial
Stack	push / pop / peek	$O(1)$	$O(n)$
Queue	enqueue / dequeue	$O(1)$	$O(n)$
Circular Queue	enqueue / dequeue	$O(1)^*$	$O(n)$
First Fit	allocate	$O(n)$	$O(1)$
Best Fit	allocate	$O(n)$	$O(1)$
Worst Fit	allocate	$O(n)$	$O(1)$
Coalescencia	coalesce	$O(n)$	$O(1)$
Compactación	compact	$O(n \log n)$	$O(n)$
FIFO Scheduler	add / next	$O(1)$	$O(n)$
Round Robin	tick (un proceso)	$O(1)$	$O(n)$
SJF	add / next	$O(n)/O(1)$	$O(n)$
Fuerza Bruta	búsqueda	$O(n)$	$O(1)$
Greedy (Bubble)	ordenar	$O(n^2)$	$O(1)$
Backtracking	exploración	$O(n!)$	$O(n)$
Merge Sort (D&V)	ordenar	$O(n \log n)$	$O(n)$
Binary Search	buscar	$O(\log n)$	$O(1)$
DP Top-Down	asignación óptima	$O(nM)$	$O(nM)$
DP Bottom-Up	asignación óptima	$O(nM)$	$O(nM)$
Incremental (sort)	inserción ordenada	$O(n^2)$	$O(1)$

* amortizado; expansión dinámica ocasional $O(n)$. M = memoria total.

❖ Recurrencias formales

Métodos utilizados

- Sustitución – Cola y Pila
$$T(n) = T(n-1) + c \rightarrow T(n) = nc + T(0) = O(n)$$
- Árbol de recurrencia – Merge sort
$$T(n) = 2T(n/2) + n$$
 - Nivel 0: n trabajo
 - Nivel 1: $2 \times n/2 = n$ trabajo
 - Nivel k: $2^k \times n/2^k = n$ trabajo.
 - Profundidad del árbol: $\log_2 n$.

Por lo tanto, $T(n) = n * \log_2 n = O(n \log n)$

- Recurrencia no homogénea – Insertion Sort
$$T(n) = T(n-1) + n \quad T(1) = 1$$

Solución por telescópica:

$$T(n) = n + (n-1) + \dots + 1 = n(n+1)/2 = O(n^2)$$
- Recurrencia Backtracking
$$T(n) = n * T(n-1) + O(1)$$

Expansión:

$$T(n)n = n * (n-1) * T(n-2) + \dots = n! * T(0) = O(n!)$$

❖ Integración C/Python

El flujo de integración sigue el modelo propuesto en el PDF:

1. Python genera procesos (generate_processes.py): crea archivos CSV con pid, burst, memory aleatorios.
2. C ejecuta la simulación (bin/main): lee los procesos, los planifica y administra memoria.
3. C exporta resultados (utils/parser.c): escribe tiempos y estadísticas en reporte/cv/.
4. Python analiza y grafica ([benchmark.py](#) + [graphs.py](#)): lee los CSV y genera gráficas en PNG/PDF.

```

1 for size in SIZES:
2     times = []
3     for _ in range(REPS):          # 3 repeticiones para reducir
        varianza
4         start = time.perf_counter()
5         subprocess.run([BINARY], capture_output=True)
6         elapsed_ms = (time.perf_counter() - start) * 1000
7         times.append(elapsed_ms)
8
9     results.append({
10         "size": size,
11         "avg_ms": sum(times) / len(times),
12         "min_ms": min(times),
13         "max_ms": max(times),
14     })

```

Lás gráficas generadas ([graphs.py](#)) incluyen:

- Tiempo de ejecución vs tamaño de entrada.
- Comparación de estrategias de memoria.
- Comparación de schedulers (tiempo de espera promedio).
- Curvas de complejidad $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.
- Diagrama de Gantt de Round Robin.

❖ Prueba y validación

La suite de tests/cubre los seis componentes principales:

Archivo	Función probada	Resultado
test_stack.c	push, pop, is_empty	PASS
test_queue.c	enqueue, dequeue (FIFO)	PASS
test_circular_queue.c	comportamiento RR	PASS
test_memory_manager.c	first_fit, free	PASS
test_scheduler.c	FIFO scheduler	PASS

Todos los tests usan `assert()` estrictos. La compilación con `-Wall` Wextra no produce ningún warning en el código de producción.

El test de memoria verifica la reutilización de bloques liberados:

```

1 MemoryManager* mm = mm_create(1024);
2 int block1 = mm_allocate_first_fit(mm, 100);
3 int block2 = mm_allocate_first_fit(mm, 200);
4 assert(block1 >= 0);
5 assert(block2 >= 0);
6 mm_free(mm, block1);
7 int block3 = mm_allocate_first_fit(mm, 50);
8 assert(block3 >= 0);    // reutiliza el hueco de block1
9 mm_destroy(mm);

```

❖ Compilación y ejecución

#Compilar el proyecto completo

make

#Ejecutar el simulador

./bin/main

#Generar benchmarking (requiere Python 3 + pandas +matplotlib)

python3 scripts/[benchmark.py](#)

Generar todas las graficas

python3 scripts/[graphs.py](#)

#Validar resultados

python3 scripts/validate_results.py

Conclusiones

El proyecto demuestra la integración de estructuras de datos y algoritmos en un contexto aplicado al dominio de sistemas operativos:

1. Estructuras de datos: pila, cola, cola circular y lista doblemente ligada fueron implementadas desde cero en C, con complejidad $O(1)$ para las operaciones fundamentales.
2. Algoritmos: se implementaron seis paradigmas (fuerza bruta, greedy, backtracking, divide y vencerás, Dp top-down y bottom-up, incremental), cada uno aplicado a un problema concreto del simulador.
3. Complejidad: cada componente incluye análisis formal con recurrencias resueltas por sustitución, árbol de recurrencia, telescopía y Teorema Maestro.
4. Integración C/Python: Python genera datos de prueba a analiza resultados; C ejecuta la simulación. El flujo es completamente automatizable con make benchmark.

5. Calidad técnica: compilación limpia con `-Wall -Wextra`, suite de tests con `assert ()`, y validación automatizada en Python.